

El uso de tuberías nos permite redirigir la salida de un proceso a la entrada de otro.

Vamos a ver su funcionamiento con un ejemplo. En el apartado 5.1 de los apuntes está el ejemplo:

 En el siguiente archivo encontrarás los ejemplos sobre Comunicación de procesos con Sockets y tuberías, solución para NetBeans

(0.05 MB)

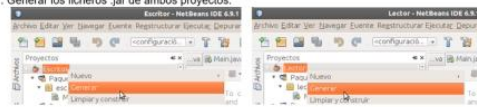
 Para saber más

Comunicación entre procesos

Probar nuestro ejemplo de tuberías

Para probar nuestro ejemplo con los procesos que lo forman, haremos lo siguiente:

1. Generar los ficheros .jar de ambos proyectos.



1. Lanzar la ejecución desde un terminal de comandos.

Para que nos sea más cómodo, hemos copiado los ficheros .jar generados en la misma carpeta.

En Windows:

```
Users\usuario\MyDesktop  
C:\Users\usuario> java -jar Escritor.jar | java -jar Lector.jar  
Proceso lector
```



1:35 / 1:50

Hay que construir (Build) los proyectos, Eso hace dentro que se genere la carpeta "dist" con los archivos "Lector.jar" y "Escritor.jar" respectivamente, en su interior.

Copio esos archivos a la carpeta D:/syp (por pura comodidad a la hora de escribir comandos en la consola)

Ejecuto Escritor y se observa como escribe 10 números en consola (salida estándar).

Si Ejecuto Escritor | Lector, se ve como es el proceso Lector el que escribe en consola su nombre y a continuación lo que ha leído como resultado del proceso Escritor:

```
Administrador: Símbolo del sistema
D:\syp>java -jar Escritor.jar
0
1
2
3
4
5
6
7
8
9

D:\syp>java -jar Escritor.jar | java -jar Lector.jar
Proceso lector
0
1
2
3
4
5
6
7
8
9

D:\syp>
```

Para el primer ejercicio de la tarea, me he basado en ese ejemplo.

Cuando ejecuto aleatorios obtengo una secuencia de números aleatorios en pantalla:

```
Símbolo del sistema
c:\syp\UD1>java -jar aleatorios.jar
401
598
817
324
365
150
111
519
188
714
629
384
139

c:\syp\UD1>
```

Ahora le añado un pipe (tubería) a la salida de aleatorios para que el proceso ordenarNumeros los coja como entrada y los ordene antes de mostrarlos:

```
Símbolo del sistema
c:\syp\UD1>java -jar aleatorios.jar | java -jar ordenarNumeros.jar
Proceso ordenaNúmeros
Los enteros ordenados son:
970
964
845
826
817
811
764
749
732
663
582
523
483
443
431
361
255
213
155
152
141
127
122
89
32
31
Número de elementos leídos: 26
c:\syp\UD1>
```

Evidentemente la cantidad de números y los valores de la anterior captura y esta son diferentes, ya que se generan de forma aleatoria en cada ejecución.